ORDER DELIVERY TIME PREDICTION

Import essential libraries for data manipulation and analysis

```
import os
import sys
import warnings
warnings.filterwarnings("ignore")

import numpy as np
import pandas as pd
from datetime import datetime
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, cross_val_score, Grid
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler, FunctionTr
from sklearn.linear_model import LinearRegression, RidgeCV, LassoCV, Ridge,
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_sco
from sklearn.impute import SimpleImputer
import joblib

print("Python", sys.version)
print("Libraries loaded.")
```

```
Python 3.12.11 (main, Jun  4 2025, 08:56:18) [GCC 11.4.0]
Libraries loaded.
```

1.Loading the data

```
# 1. Load the data
possible = [
    '/content/porter_data_1.csv',
    '/mnt/data/porter_data_1.csv',
    './porter_data_1.csv'
]
csv_path = None
for p in possible:
    if os.path.exists(p):
        csv_path = p
        break
if csv_path is None:
    raise FileNotFoundError('porter_data_1.csv not found. Upload the CSV to

df = pd.read_csv(csv_path)
print('Loaded', csv_path, 'with shape', df.shape)
df.head()
```

```
Loaded /content/porter_data_1.csv with shape (175777, 14)
```

	market_id	created_at	actual_delivery_time	store_primary_category	order
0	1.0	2015-02-06 22:24:17	2015-02-06 23:11:17	4	
1	2.0	2015-02-10 21:49:25	2015-02-10 22:33:25	46	
2	2.0	2015-02-16 00:11:35	2015-02-16 01:06:35	36	
3	1.0	2015-02-12 03:36:46	2015-02-12 04:35:46	38	
4	1.0	2015-01-27 02:12:36	2015-01-27 02:58:36	38	

2.Data Preprocessing and Feature Engineering

2.1 Fixing the Datatypes

```
# 2. Data Preprocessing and Feature Engineering
# 2.1 Fixing the datatypes

# 2.1.1 Convert created_at & actual_delivery_time to datetime (2 marks)
for col in ['created_at', 'actual_delivery_time']:
    if col in df.columns:
        df[col] = pd.to_datetime(df[col], infer_datetime_format=True, error

print('Datetime conversion done. Nulls after conversion:')
print(df[['created_at','actual_delivery_time']].isnull().sum())

# 2.1.2 Convert categorical fields to category dtype (3 marks)
cat_cols = []
```

```
for c in ['store_primary_category','order_protocol','market_id']:
    if c in df.columns:
        df[c] = df[c].astype('category')
        cat_cols.append(c)
print('Converted to category:', cat_cols)
df.dtypes
```

```
Datetime conversion done. Nulls after conversion:
created_at              0
actual_delivery_time    0
dtype: int64
Converted to category: ['store_primary_category', 'order_protocol', 'market_
```

	0
market_id	category
created_at	datetime64[ns]
actual_delivery_time	datetime64[ns]
store_primary_category	category
order_protocol	category
total_items	int64
subtotal	int64
num_distinct_items	int64
min_item_price	int64
max_item_price	int64
total_onshift_dashers	float64
total_busy_dashers	float64
total_outstanding_orders	float64
distance	float64

dtype: object

```
# 2.2 Feature Engineering (5 marks)

# 2.2.1 Calculate time taken (2 marks)
df['time_taken'] = (df['actual_delivery_time'] - df['created_at']).dt.total

# Basic cleanup: remove non-positive or null durations
before = df.shape[0]
df = df[df['time_taken'].notnull() & (df['time_taken'] > 0)].copy()
after = df.shape[0]
print(f'Removed {before-after} rows with invalid durations. New shape: {df.

# 2.2.2 Extract hour and day, create isWeekend, drop unnecessary columns (3
df['order_hour'] = df['created_at'].dt.hour
df['order_dayofweek'] = df['created_at'].dt.dayofweek  # Monday=0
df['isWeekend'] = df['order_dayofweek'].isin([5,6]).astype(int)
```

```
# Drop columns not needed for modelling (keep originals if you prefer)
drop_cols = ['created_at','actual_delivery_time']  # preserve only engineer
for c in drop_cols:
    if c in df.columns:
        df.drop(columns=c, inplace=True)

print('Engineered time features. Columns now:', df.columns.tolist())
```

```
Removed 0 rows with invalid durations. New shape: (175777, 15)
Engineered time features. Columns now: ['market_id', 'store_primary_category
```

```
# 2.3 Creating training and validation sets (5 marks)

# 2.3.1 Define target and features (2 marks)
target = 'time_taken'  # minutes
exclude = [target]
features = [c for c in df.columns if c not in exclude]
print('Number of features available:', len(features))

# 2.3.2 Split into train and test (3 marks)
X = df[features].copy()
y = df[target].copy()

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, ra
print('Train shape:', X_train.shape, 'Test shape:', X_test.shape)
```

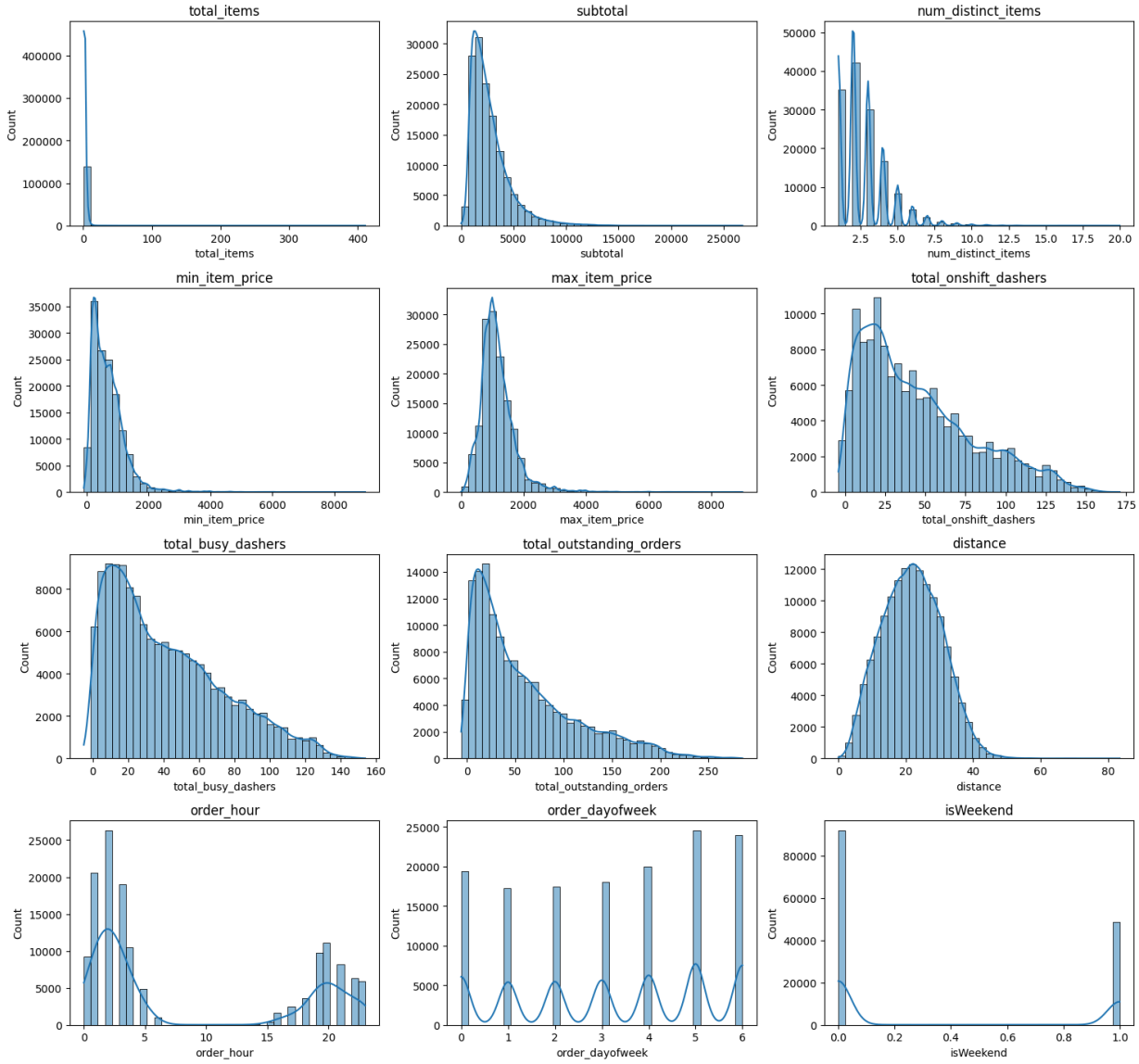
```
Number of features available: 15
Train shape: (140621, 15) Test shape: (35156, 15)
```

```
# 3. Exploratory Data Analysis on Training Data (20 marks)
# 3.1 Feature Distributions (7 marks)

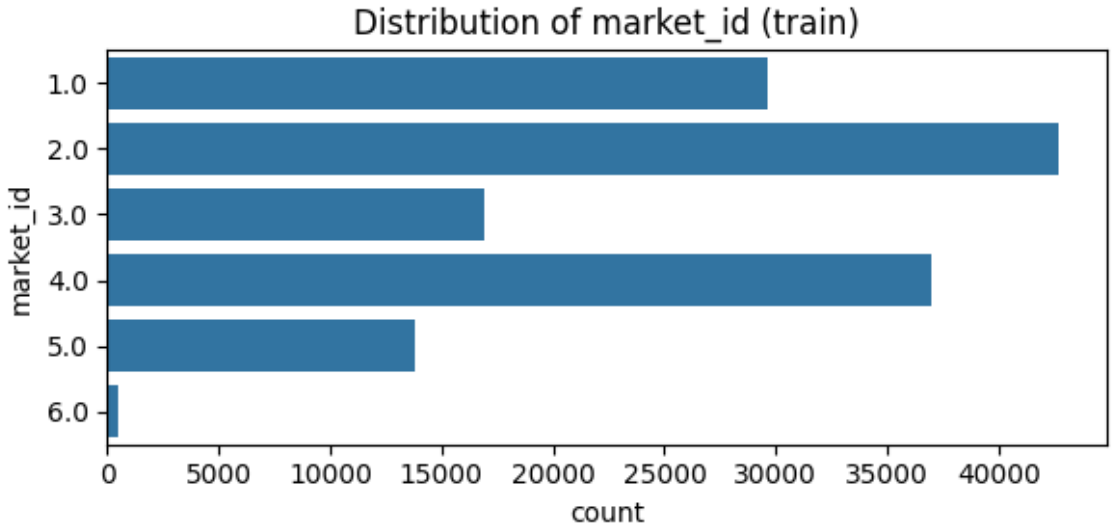
numeric_cols = X_train.select_dtypes(include=[np.number]).columns.tolist()
categorical_cols = X_train.select_dtypes(include=['category','object']).col
print('Numeric columns:', numeric_cols)
print('Categorical columns:', categorical_cols)
```

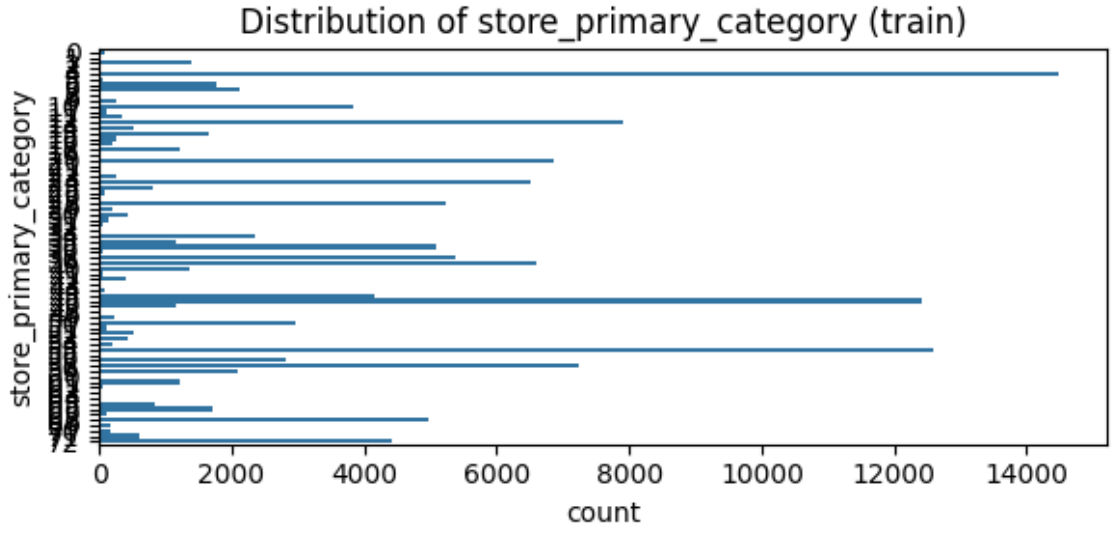
```
Numeric columns: ['total_items', 'subtotal', 'num_distinct_items', 'min_item
Categorical columns: ['market_id', 'store_primary_category', 'order_protocol
```

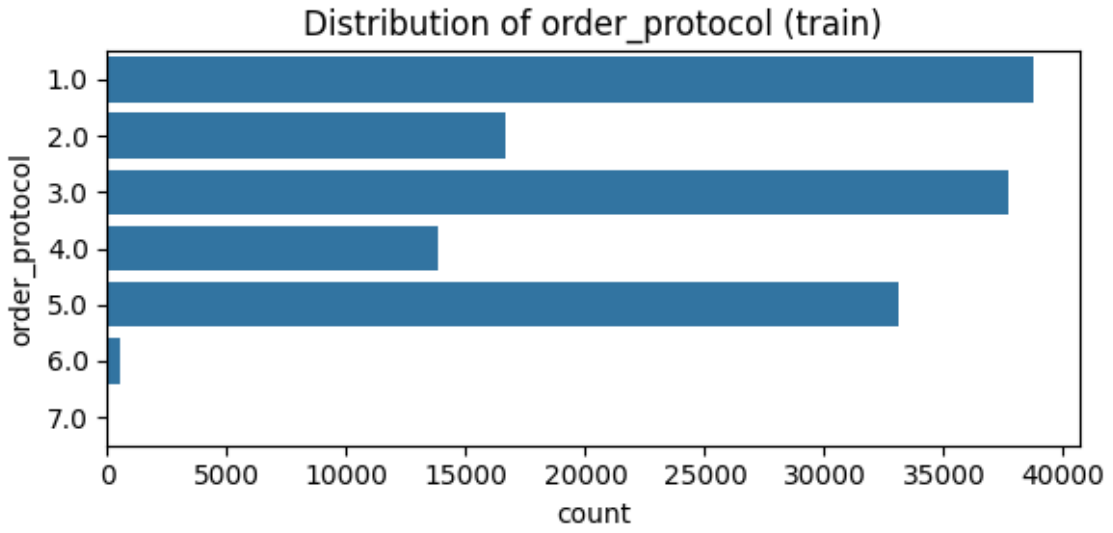
```
# 3.1.1 Plot distributions for numerical columns (3 marks)
import math
n = len(numeric_cols)
cols = 3
rows = math.ceil(n/cols)
plt.figure(figsize=(cols*5, rows*3.5))
for i, col in enumerate(numeric_cols, 1):
    plt.subplot(rows, cols, i)
    sns.histplot(X_train[col].dropna(), bins=40, kde=True)
    plt.title(col)
plt.tight_layout()
plt.show()
```




```
# 3.1.2 Distribution of categorical features (2 marks)
for c in categorical_cols:
    plt.figure(figsize=(6,3))
    sns.countplot(y=X_train[c])
    plt.title(f'Distribution of {c} (train)')
    plt.tight_layout()
    plt.show()
```



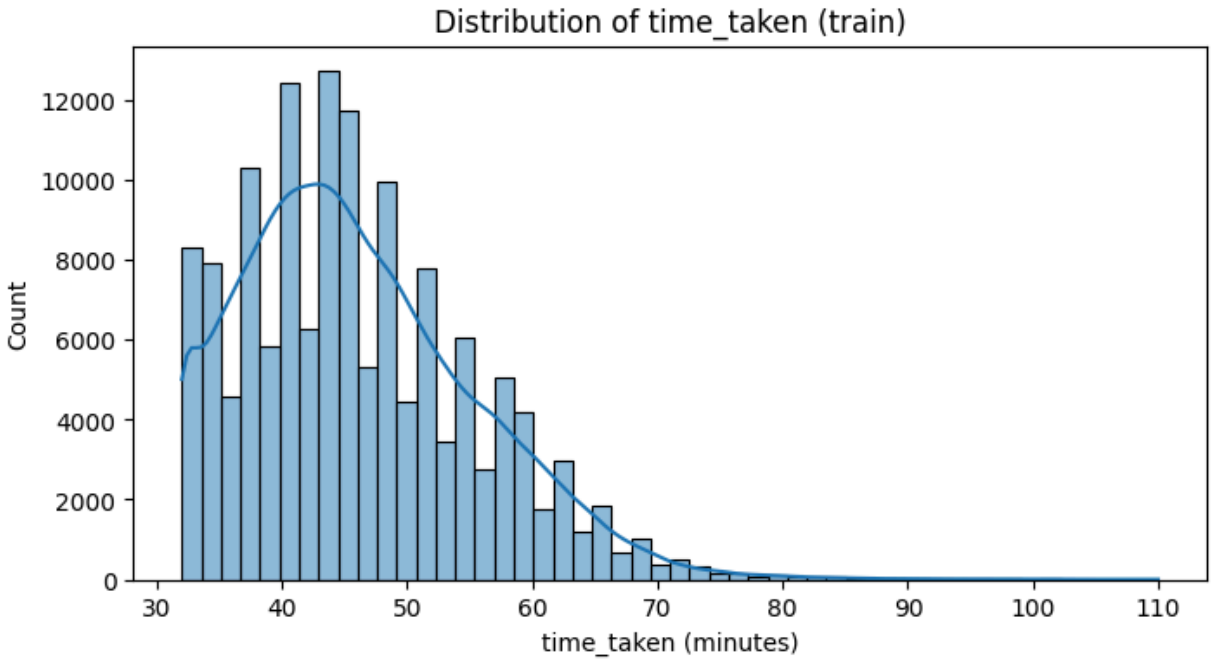


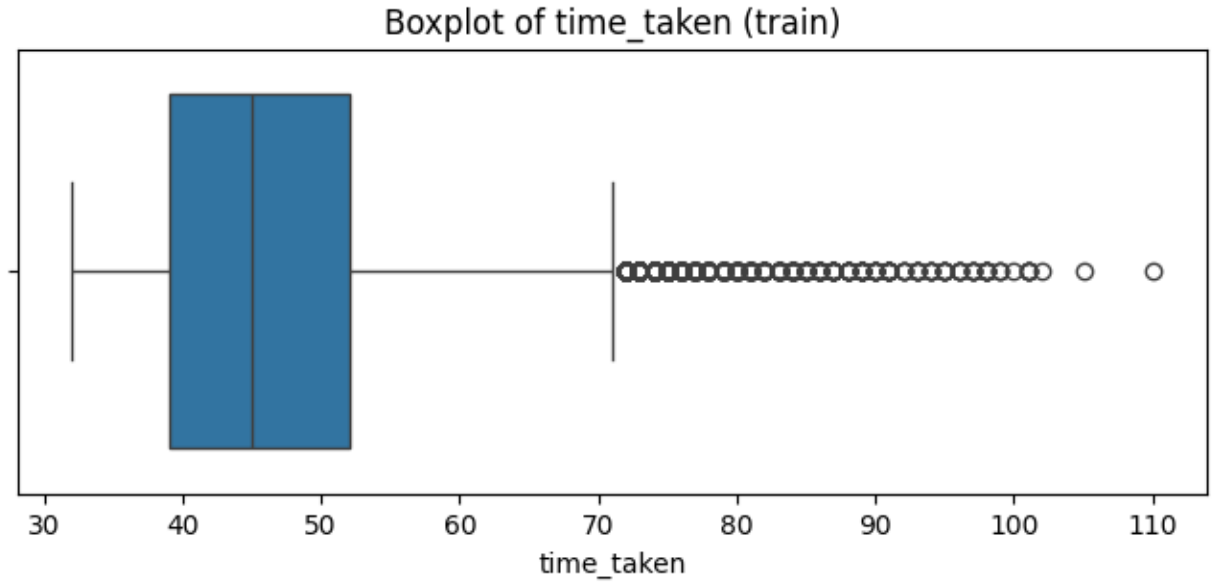



```
# 3.1.3 Distribution of target variable (2 marks)
plt.figure(figsize=(8,4))
sns.histplot(y_train, bins=50, kde=True)
plt.xlabel('time_taken (minutes)')
plt.title('Distribution of time_taken (train)')
plt.show()

plt.figure(figsize=(8,3))
sns.boxplot(x=y_train)
```

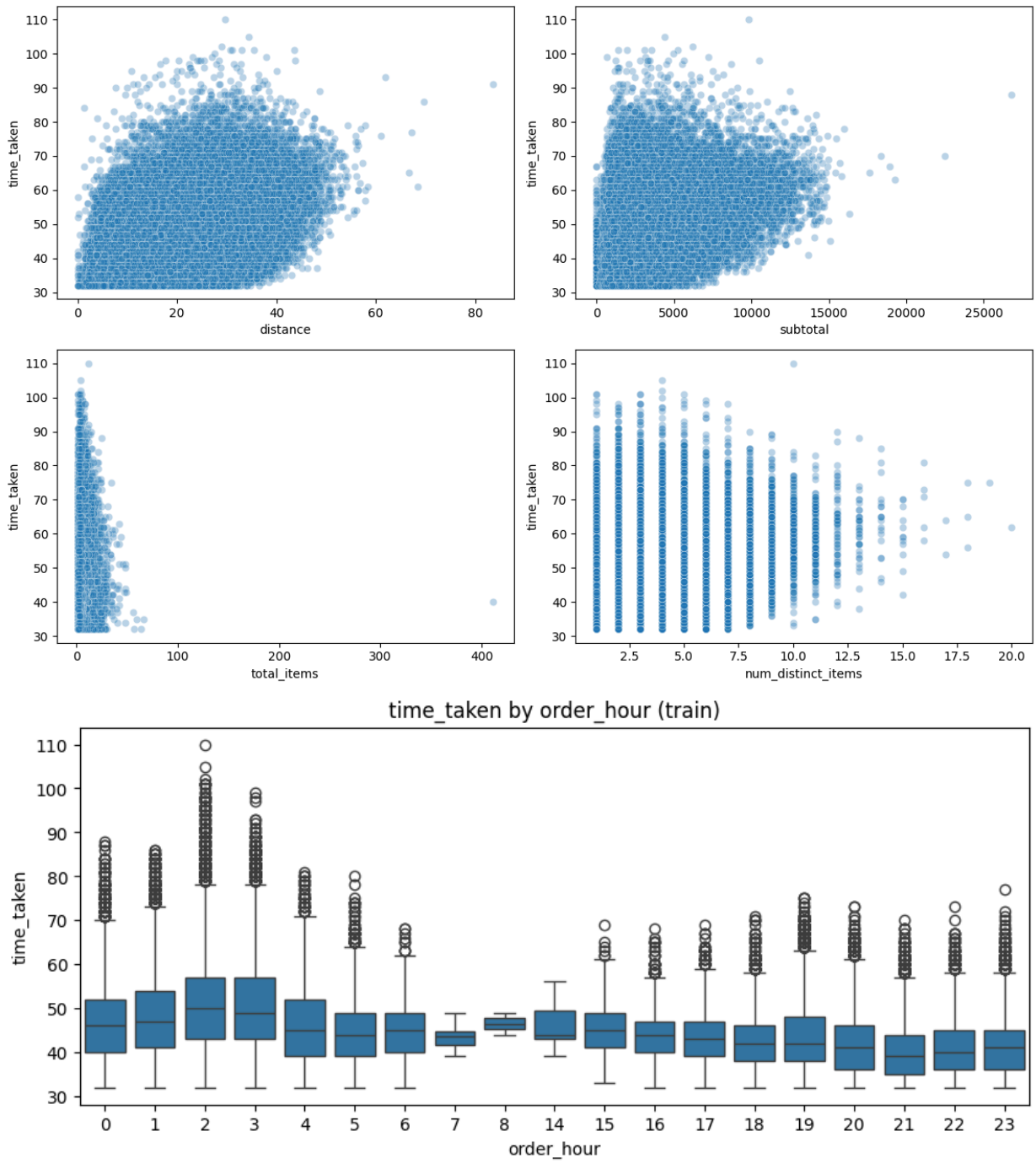
```
plt.title('Boxplot of time_taken (train)')
plt.show()
```






```
# 3.2 Relationships Between Features (3 marks)
important_num = ['distance','subtotal','total_items','num_distinct_items']
important_num = [c for c in important_num if c in X_train.columns]
plt.figure(figsize=(12,8))
for i,col in enumerate(important_num,1):
    plt.subplot(2,2,i)
    sns.scatterplot(x=X_train[col], y=y_train, alpha=0.3)
    plt.xlabel(col)
    plt.ylabel('time_taken')
plt.tight_layout()
plt.show()

if 'order_hour' in X_train.columns:
    plt.figure(figsize=(10,4))
    sns.boxplot(x=X_train['order_hour'], y=y_train)
    plt.title('time_taken by order_hour (train)')
    plt.show()
```

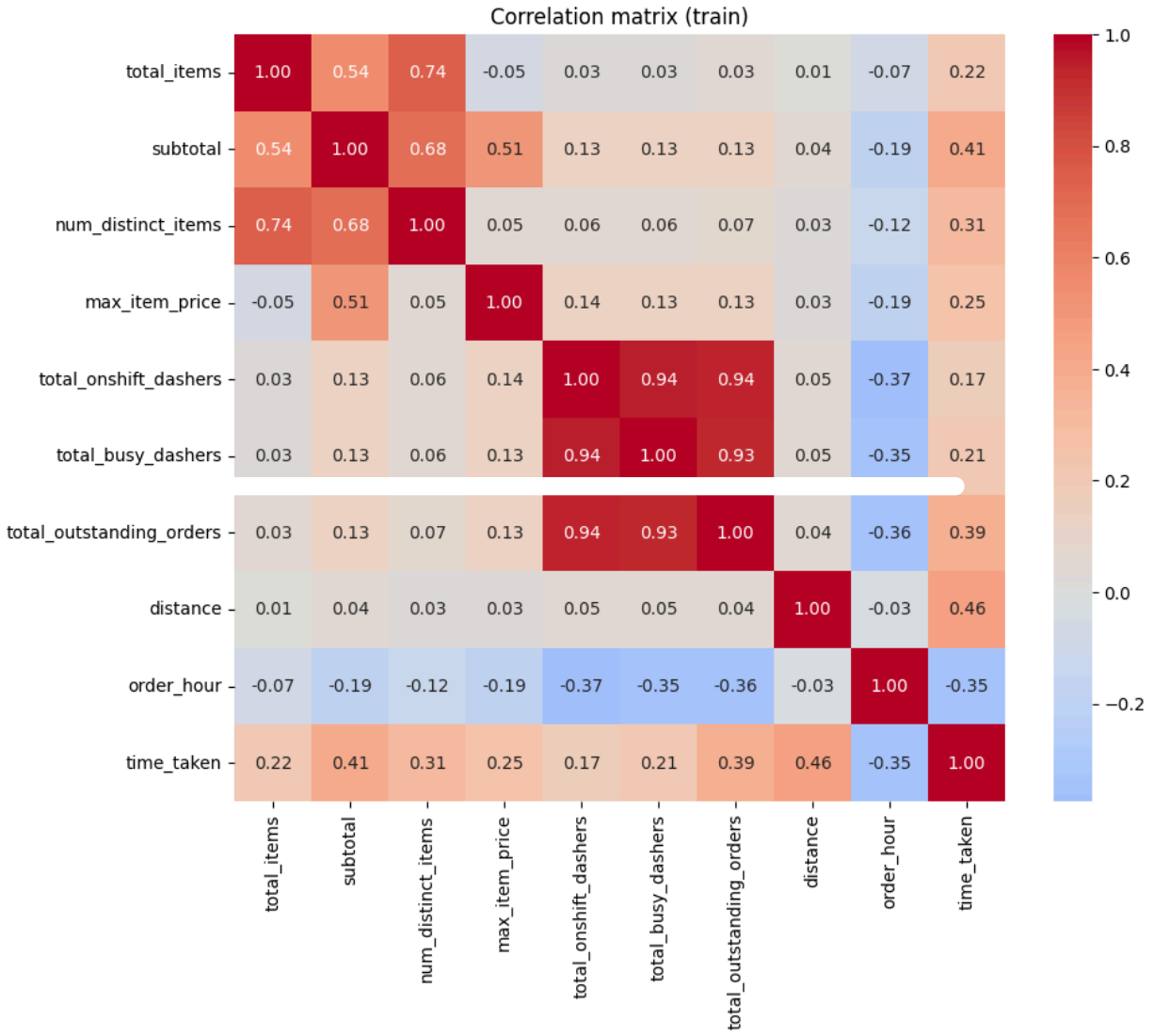



```
# 3.3 Correlation Analysis (5 marks)
plt.figure(figsize=(10,8))
corr = X_train.join(y_train).select_dtypes(include=[np.number]).corr()
sns.heatmap(corr, annot=True, fmt='.2f', cmap='coolwarm', center=0)
plt.title('Correlation matrix (train)')
plt.show()

num_corr_with_target = corr['time_taken'].drop('time_taken').abs().sort_val
print('\nNumeric absolute correlations with target (ascending):\n', num_cor

weak = num_corr_with_target[num_corr_with_target < 0.02].index.tolist()
if len(weak) < 3:
    weak = num_corr_with_target.index.tolist()[:3]
print('Dropping weakly correlated numeric columns:', weak)
```

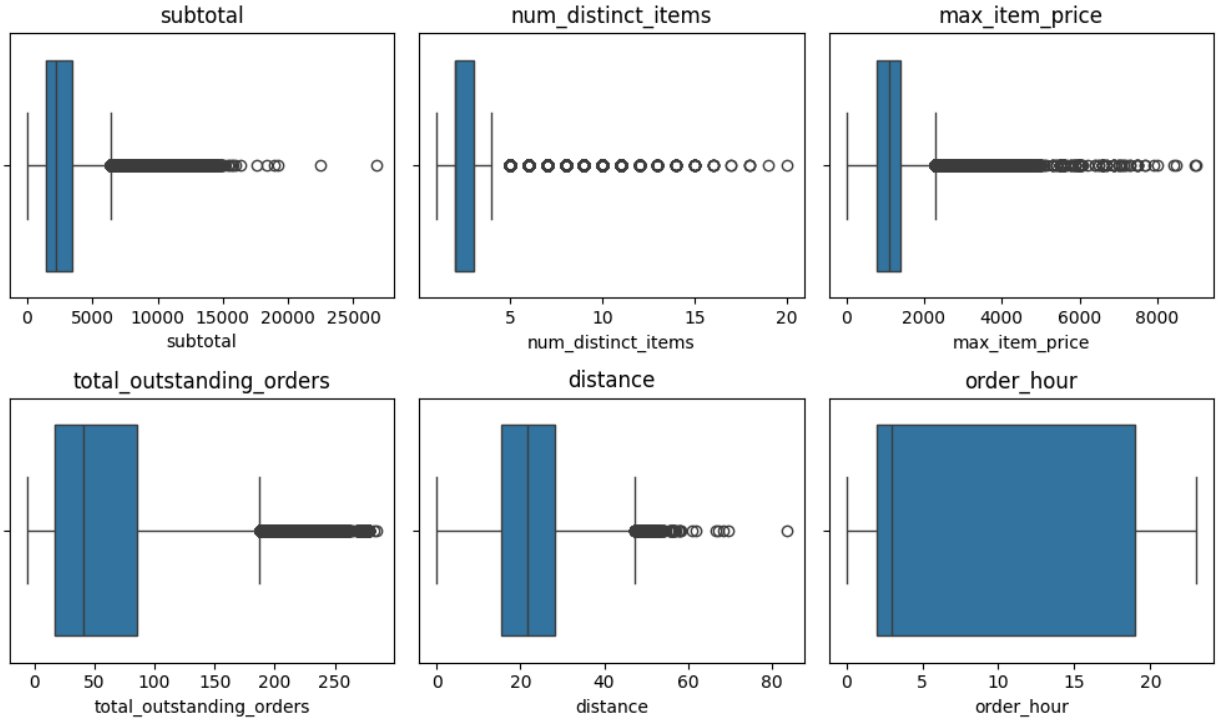
```
for c in weak:
    if c in X_train.columns:
        X_train.drop(columns=c, inplace=True)
    if c in X_test.columns:
        X_test.drop(columns=c, inplace=True)
print('New train shape after dropping weak columns:', X_train.shape)
```




```
Numeric absolute correlations with target (ascending):
 total_onshift_dashers       0.171828
total_busy_dashers           0.206514
total_items                  0.220804
max_item_price               0.254650
num_distinct_items           0.312510
order_hour                   0.345716
total_outstanding_orders     0.385028
subtotal                     0.413392
distance                     0.461026
Name: time_taken, dtype: float64
Dropping weakly correlated numeric columns: ['total_onshift_dashers', 'total
New train shape after dropping weak columns: (140621, 9)
```

```
# 3.4 Handling the Outliers (5 marks)
plt.figure(figsize=(10,6))
num_cols_plot = [c for c in numeric_cols if c in X_train.columns]
for i,col in enumerate(num_cols_plot,1):
    plt.subplot(math.ceil(len(num_cols_plot)/3),3,i)
    sns.boxplot(x=X_train[col])
    plt.title(col)
plt.tight_layout()
plt.show()

# Winsorize (clip) at 1st and 99th percentiles
from scipy.stats import mstats
num_cols_after = [c for c in num_cols_plot]
X_train_out = X_train.copy()
X_test_out = X_test.copy()
for c in num_cols_after:
    lower = X_train_out[c].quantile(0.01)
    upper = X_train_out[c].quantile(0.99)
    X_train_out[c] = X_train_out[c].clip(lower,upper)
    X_test_out[c] = X_test_out[c].clip(lower,upper)
print('Applied clipping at 1st and 99th percentiles for numeric variables.'
```




```
Applied clipping at 1st and 99th percentiles for numeric variables.
```

```
# 4. Exploratory Data Analysis on Validation Data [Optional]

# Define numerical and categorical columns for validation set
val_num_cols = X_test_out.select_dtypes(include=[np.number]).columns.tolist
val_cat_cols = X_test_out.select_dtypes(include=['object','category']).colu

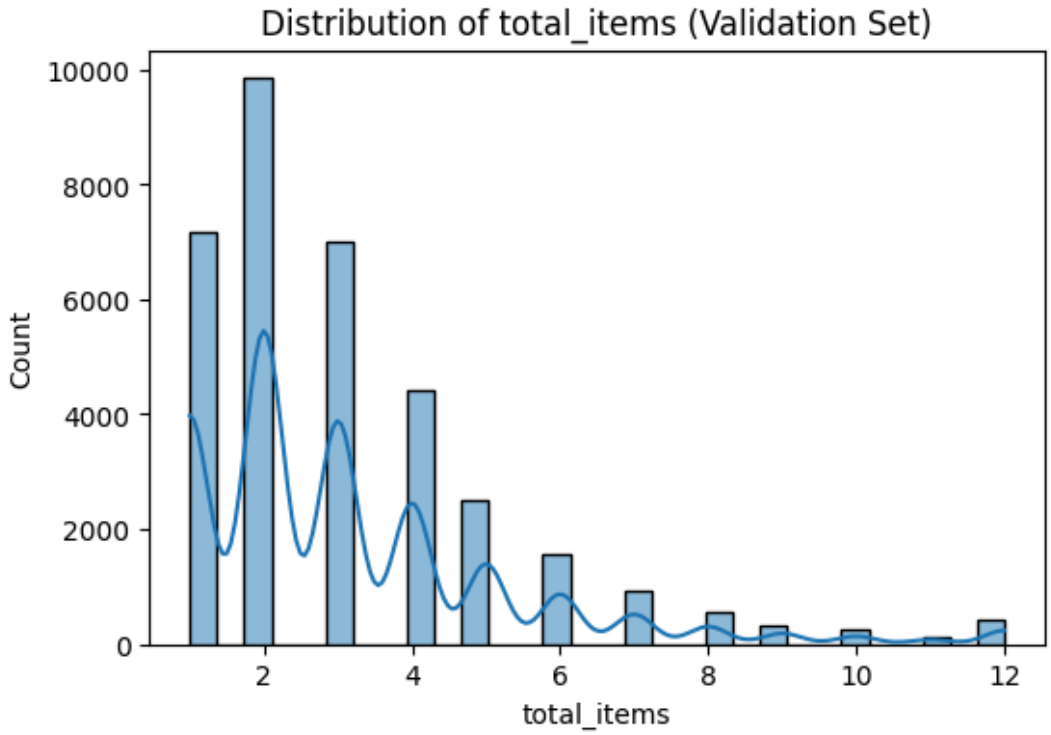
print("Validation numerical columns:", val_num_cols)
print("Validation categorical columns:", val_cat_cols)
```

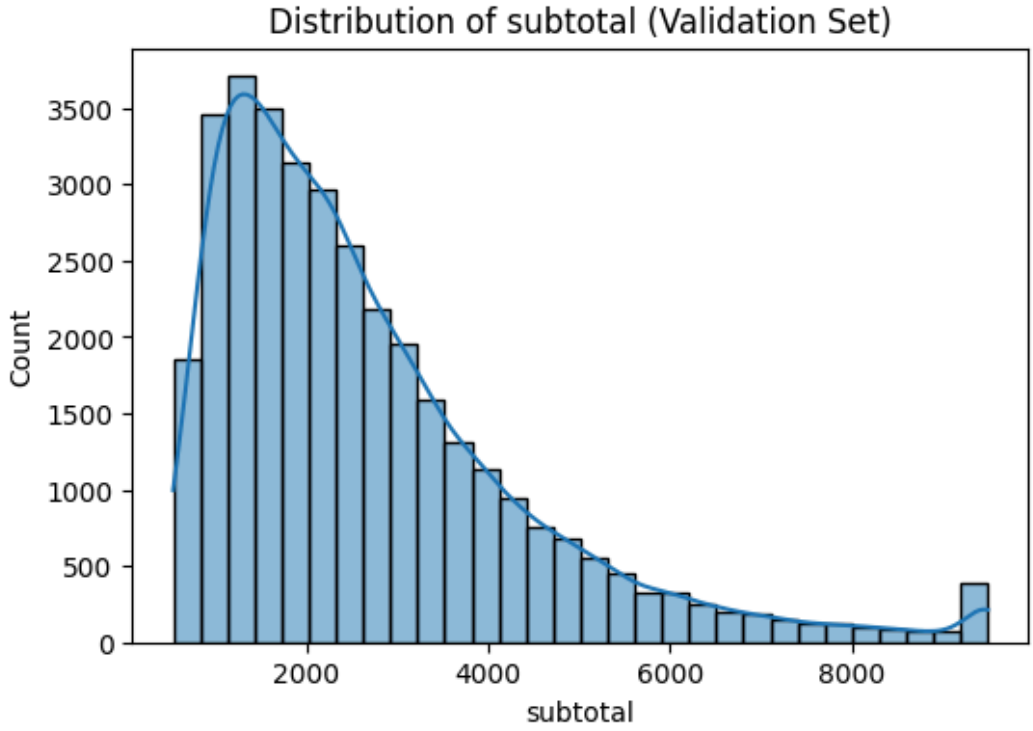
```
Validation numerical columns: ['subtotal', 'num_distinct_items', 'max_item_p
Validation categorical columns: ['market_id', 'store_primary_category', 'ord
```

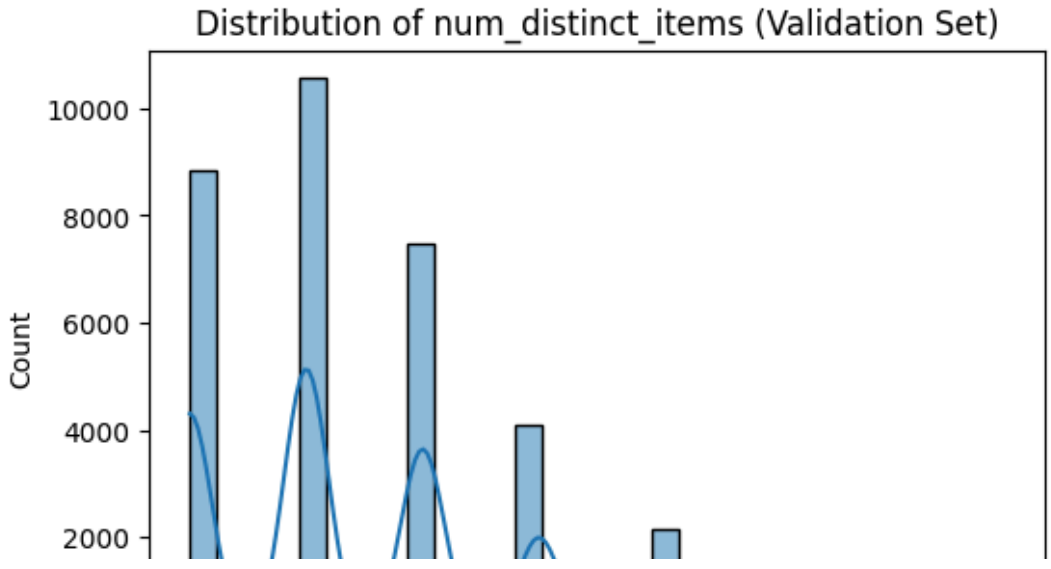
4.1 Feature Distributions

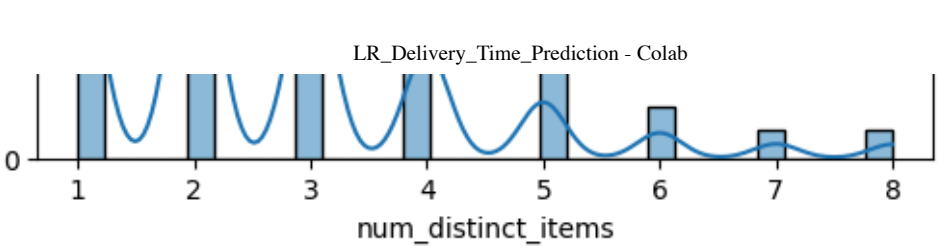
4.1.1 Plot distributions for numerical columns

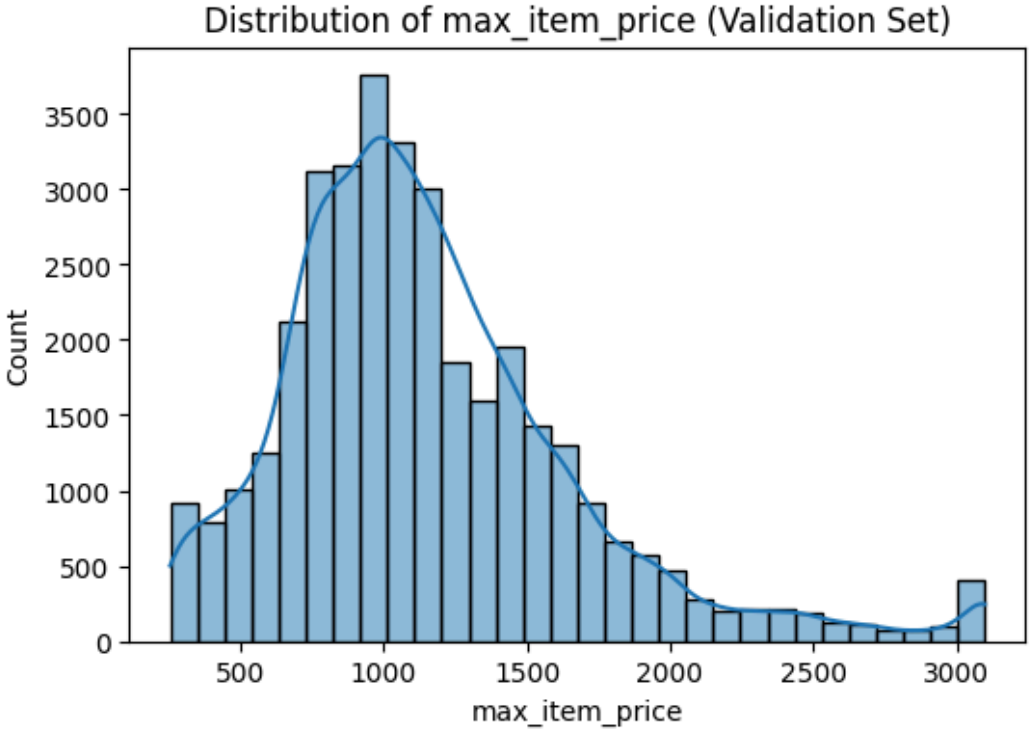
```
# Plot distributions for numerical columns in validation data
for col in val_num_cols:
    plt.figure(figsize=(6,4))
    sns.histplot(X_test_out[col], kde=True, bins=30)
    plt.title(f"Distribution of {col} (Validation Set)")
    plt.show()
```

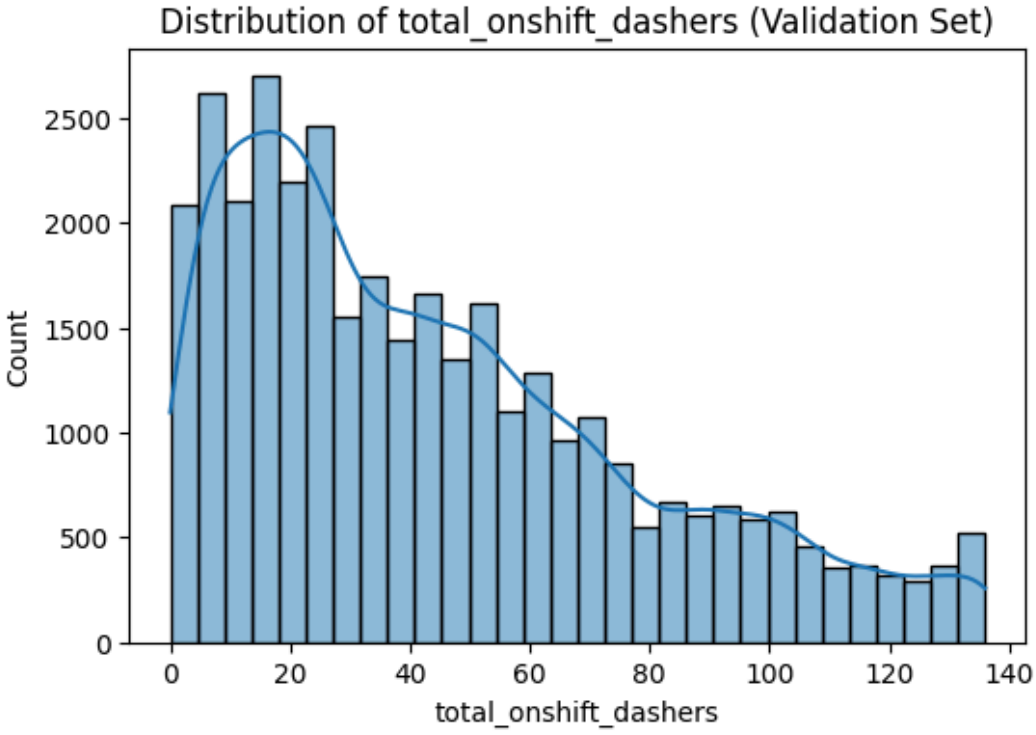

Distribution of total_items (Validation Set)
Count
total_items
0
2000
4000
6000
8000
10000
2
4
6
8
10
12

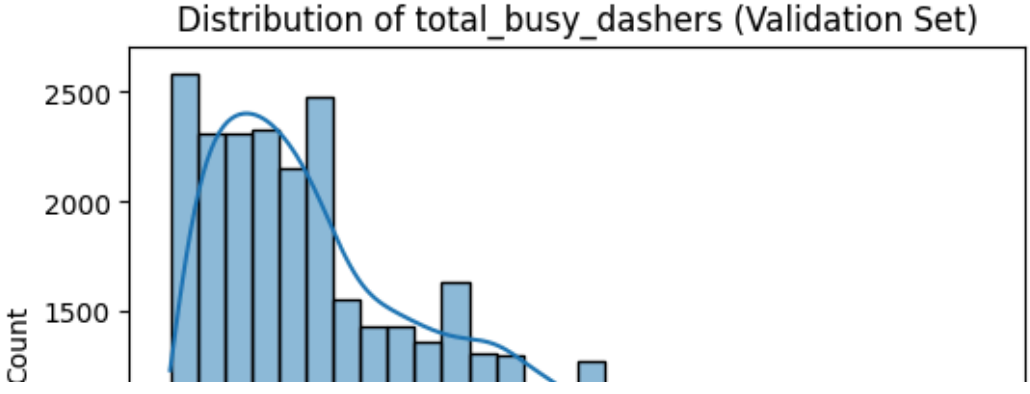

Distribution of subtotal (Validation Set)
Count
subtotal
0
500
1000
1500
2000
2500
3000
3500
2000
4000
6000
8000

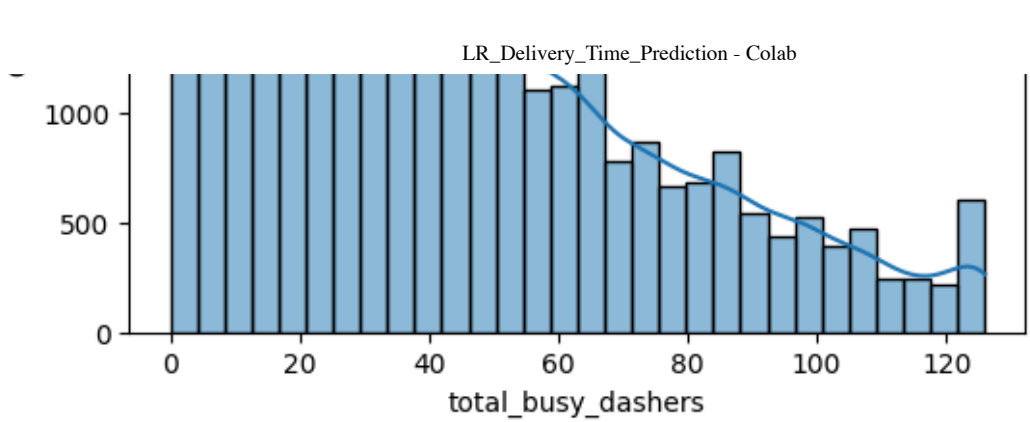

Distribution of num_distinct_items (Validation Set)
Count
2000
4000
6000
8000
10000

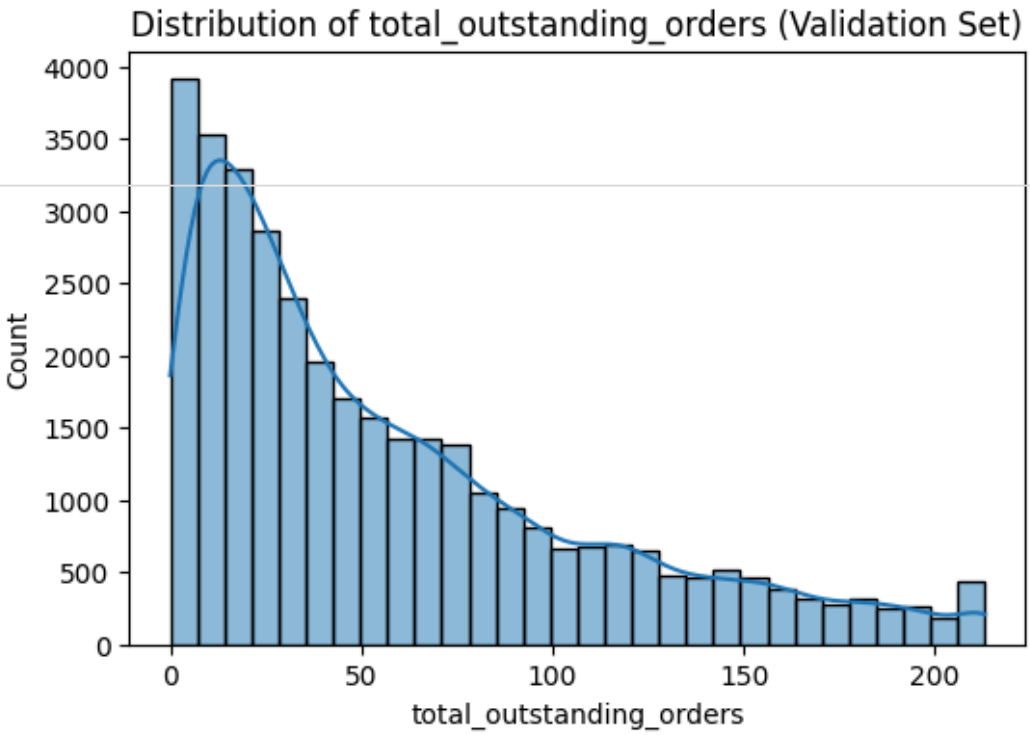

0
1
2
3
4
5
6
7
8
num_distinct_items

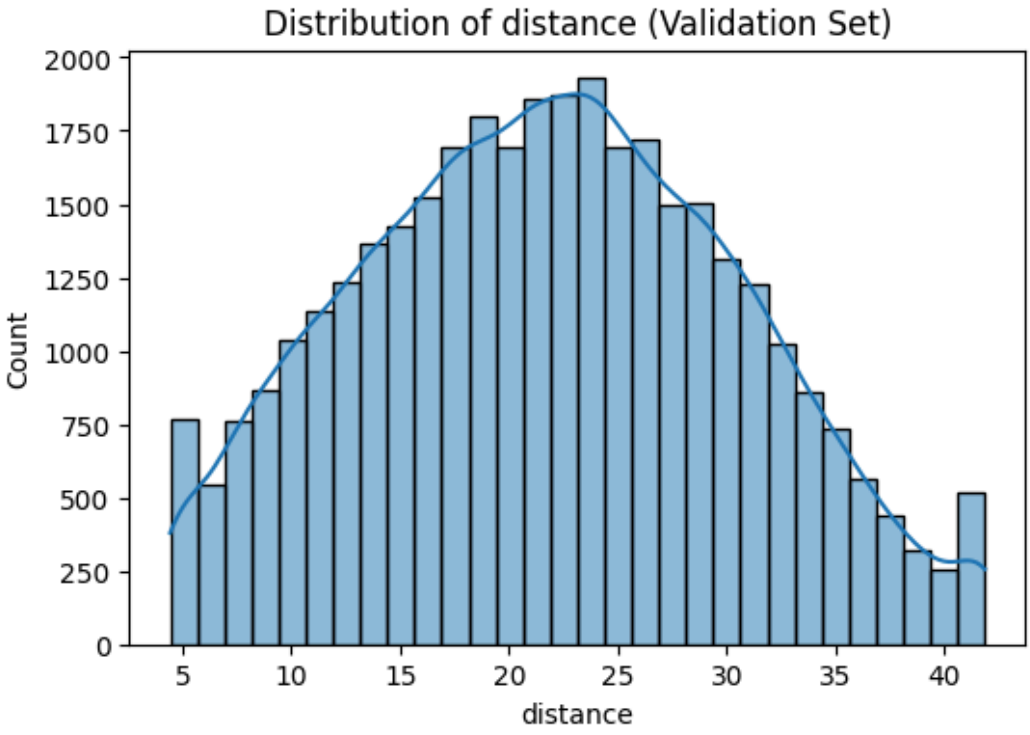

Distribution of max_item_price (Validation Set)
Count
0
500
1000
1500
2000
2500
3000
3500
max_item_price

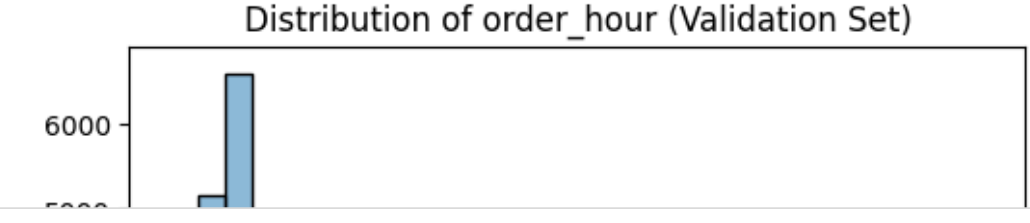

Distribution of total_onshift_dashers (Validation Set)
Count
0
500
1000
1500
2000
2500
total_onshift_dashers


Distribution of total_busy_dashers (Validation Set)
Count
1500
2000
2500


1000
500
0
0
20
40
60
80
100
120
total_busy_dashers

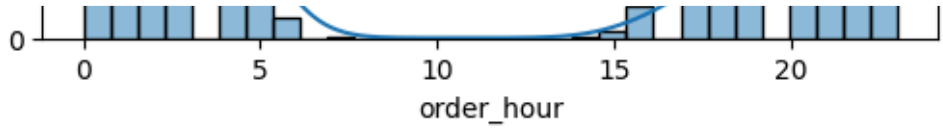

Distribution of total_outstanding_orders (Validation Set)
4000
3500
3000
2500
2000
1500
1000
500
0
Count
0
50
100
150
200
total_outstanding_orders

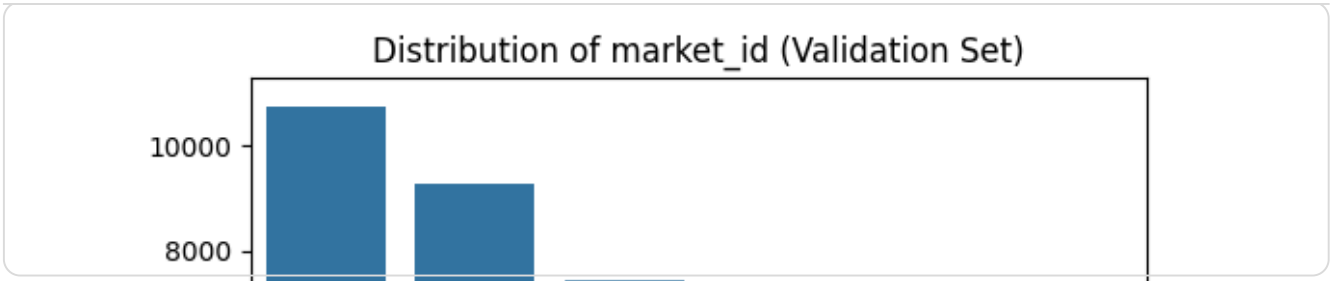

Distribution of distance (Validation Set)
2000
1750
1500
1250
1000
750
500
250
0
Count
5
10
15
20
25
30
35
40
distance


Distribution of order_hour (Validation Set)
6000

4.1.2 Check the distribution of categorical features

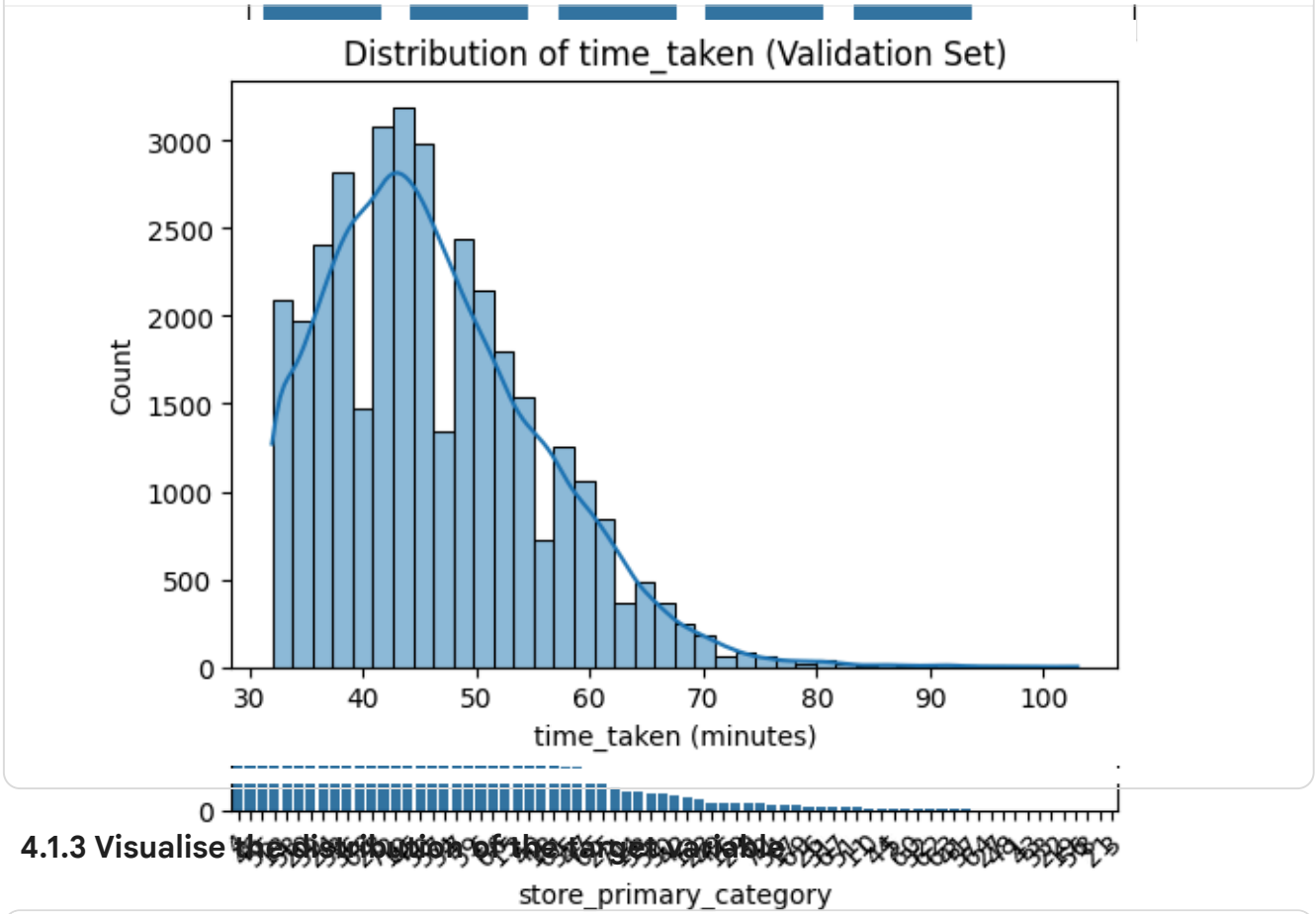
```
# Distribution of categorical columns in validation data
for col in val_cat_cols:
    plt.figure(figsize=(6,4))
    sns.countplot(data=X_test_out, x=col, order=X_test_out[col].value_count
    plt.title(f"Distribution of {col} (Validation Set)")
    plt.xticks(rotation=45)
    plt.show()
```





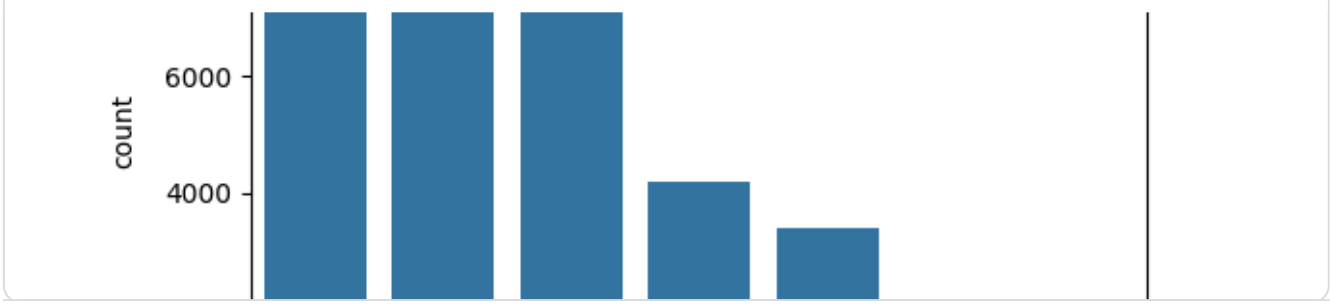
4.1.2 Check the distribution of categorical features

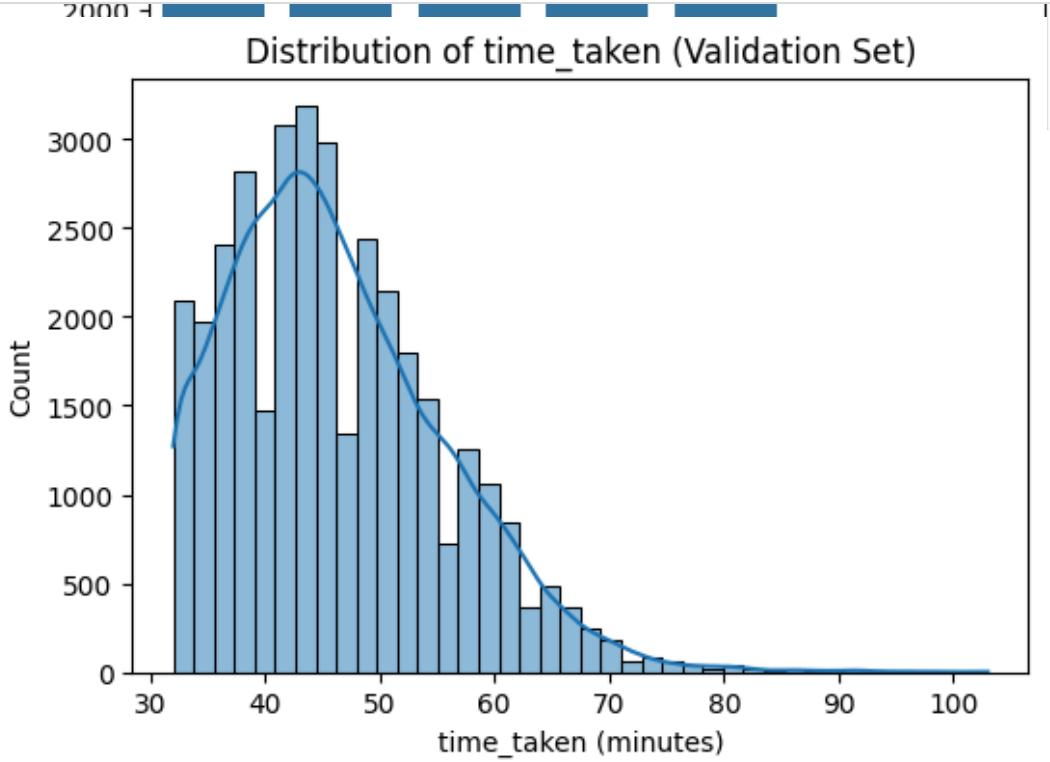
```
# Distribution of target variable in validation set
plt.figure(figsize=(6,4))
sns.histplot(y_test, bins=40, kde=True)
plt.title("Distribution of time_taken (Validation Set)")
plt.xlabel("time_taken (minutes)")
plt.show()
```



4.1.3 Visualise the distribution of the target variable


```
# Distribution of target variable in validation set
plt.figure(figsize=(6,4))
sns.histplot(y_test, bins=40, kde=True)
plt.title("Distribution of time_taken (Validation Set)")
plt.xlabel("time_taken (minutes)")
plt.show()
```

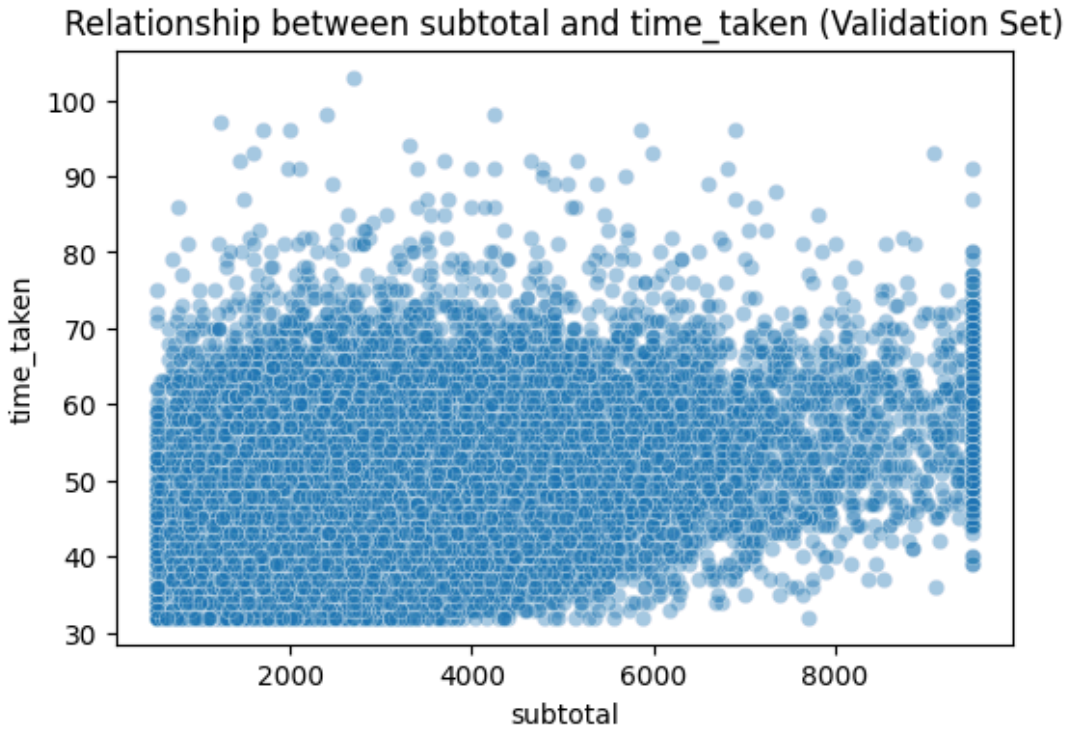


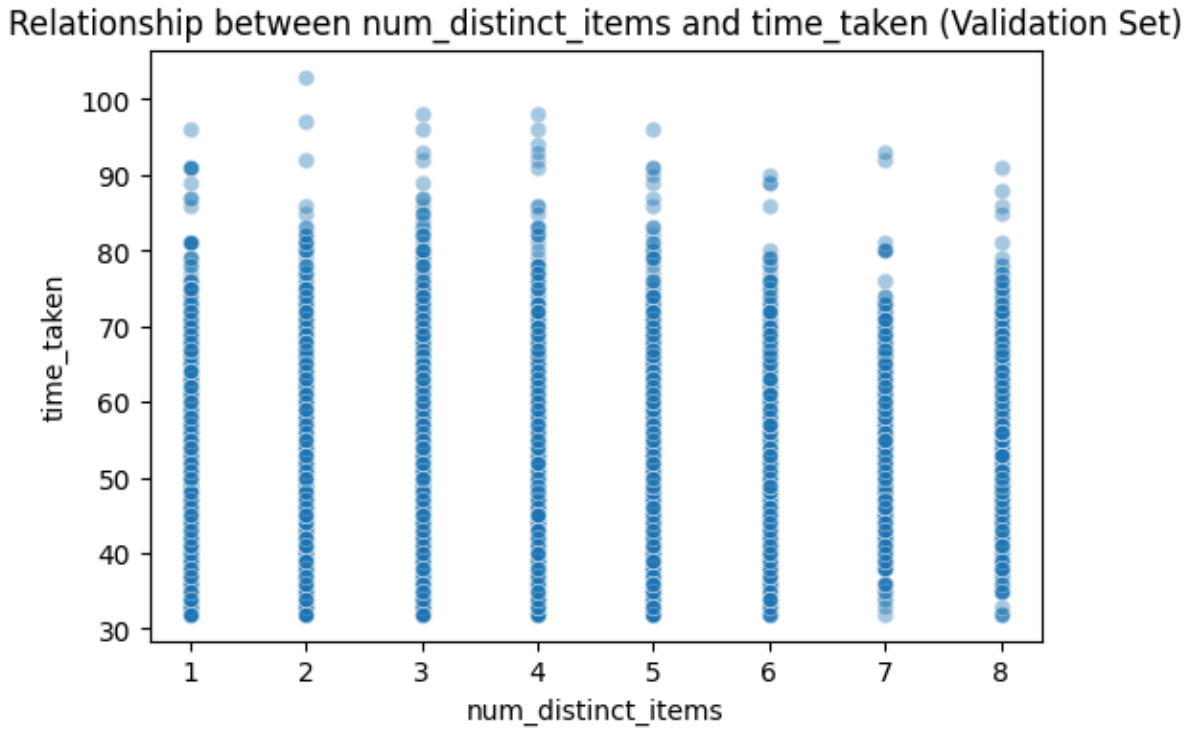


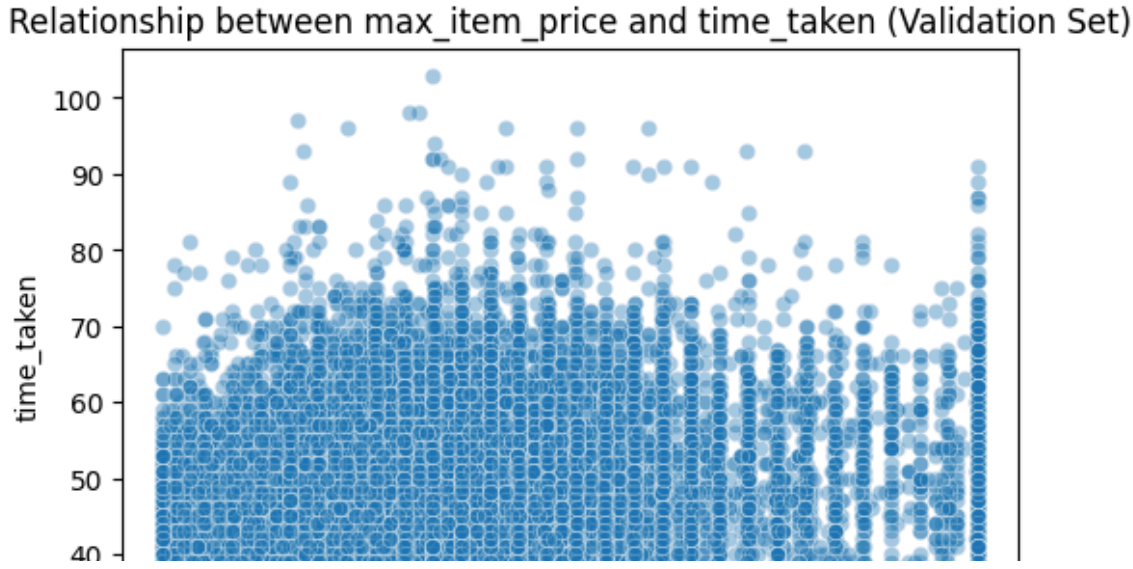
4.2 Relationships Between Features

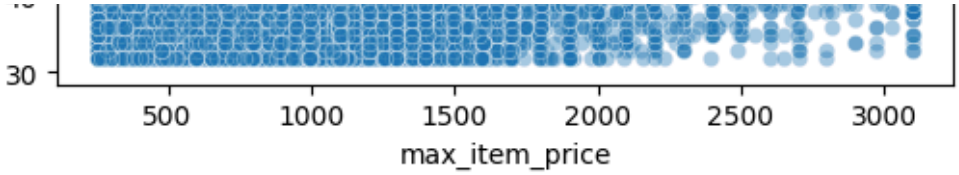
```
# Scatter plot to visualise the relationship between time_taken and numeric
for col in val_num_cols:
    plt.figure(figsize=(6,4))
    sns.scatterplot(x=X_test_out[col], y=y_test, alpha=0.4)
    plt.title(f"Relationship between {col} and time_taken (Validation Set)"
    plt.xlabel(col)
    plt.ylabel("time_taken")
    plt.show()

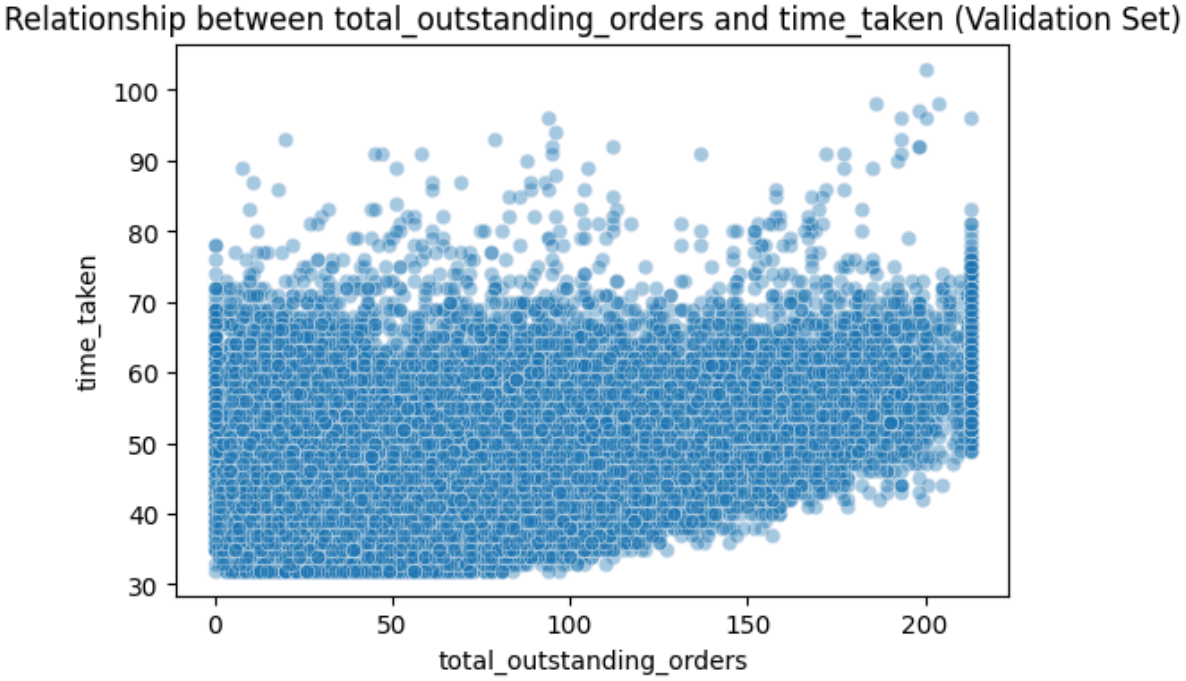
# Example: show time_taken vs order_hour if present
if 'order_hour' in X_test_out.columns:
    plt.figure(figsize=(6,4))
    sns.boxplot(x=X_test_out['order_hour'], y=y_test)
    plt.title("time_taken by Order Hour (Validation Set)")
    plt.show()
```

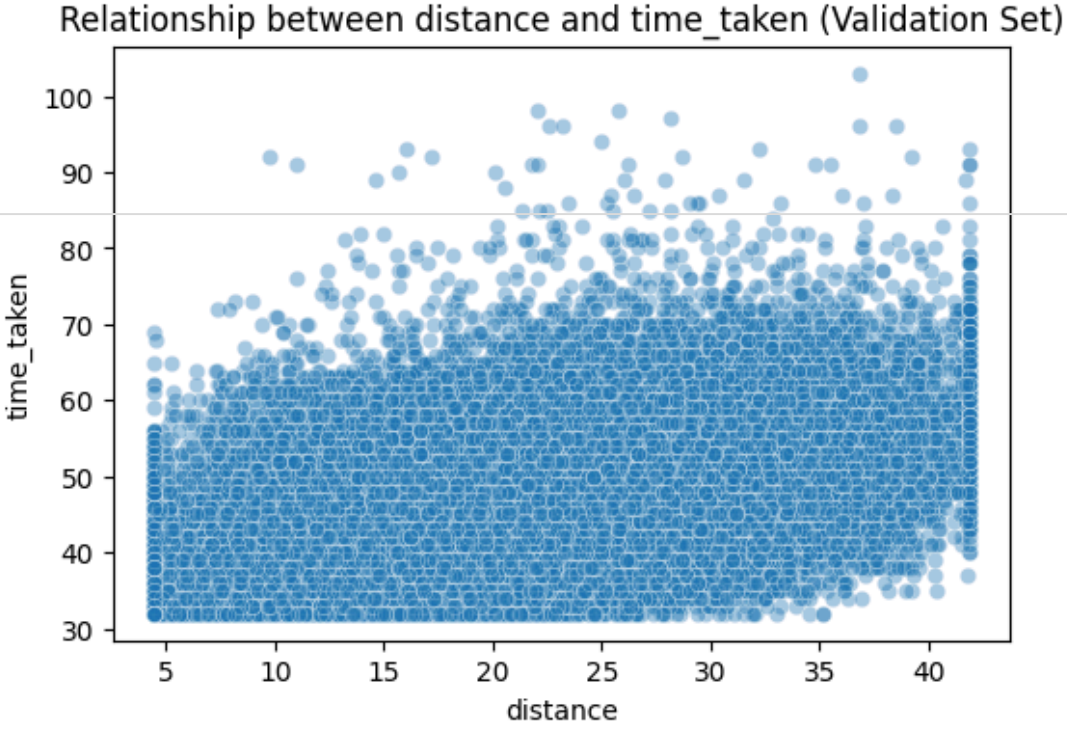

Relationship between subtotal and time_taken (Validation Set)
time_taken
subtotal
100
90
80
70
60
50
40
30
2000
4000
6000
8000

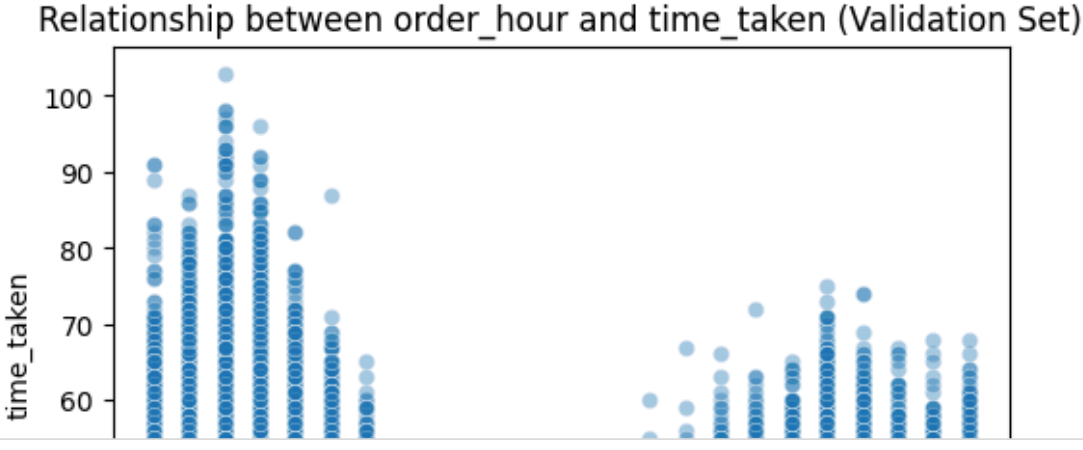

Relationship between num_distinct_items and time_taken (Validation Set)
time_taken
num_distinct_items
100
90
80
70
60
50
40
30
1
2
3
4
5
6
7
8


Relationship between max_item_price and time_taken (Validation Set)
time_taken
100
90
80
70
60
50
40


30
500
1000
1500
2000
2500
3000
max_item_price


Relationship between total_outstanding_orders and time_taken (Validation Set)
time_taken
total_outstanding_orders
30
40
50
60
70
80
90
100
0
50
100
150
200


Relationship between distance and time_taken (Validation Set)
time_taken
distance
30
40
50
60
70
80
90
100
5
10
15
20
25
30
35
40


Relationship between order_hour and time_taken (Validation Set)
time_taken
60
70
80
90
100

4.3 Drop weakly correlated columns

```
# Compute correlations of validation numerical features with target
val_corrs = pd.concat([X_test_out[val_num_cols], y_test], axis=1).corr()

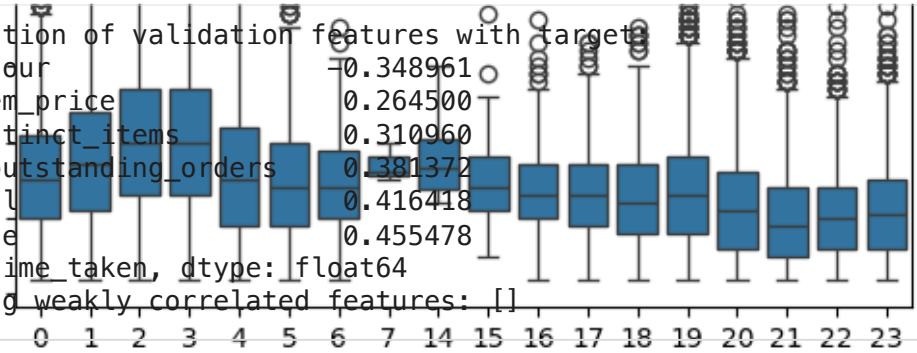
# Sort by correlation with target
target_corr = val_corrs['time_taken'].drop('time_taken').sort_values()

print("Correlation of validation features with target:")
print(target_corr)

# Drop weak correlations (< 0.05 absolute value as example)
weak_features = target_corr[abs(target_corr) < 0.05].index.tolist()
print("Dropping weakly correlated features:", weak_features)

X_test_out = X_test_out.drop(columns=weak_features)
```

```
Correlation of validation features with target:
order_hour                -0.348961
max_item_price             0.264500
num_distinct_items         0.310960
total_outstanding_orders   0.381372
subtotal                   0.416418
distance                   0.455478
Name: time_taken, dtype: float64
Dropping weakly correlated features: []
```




```
# 5. Model Building (15 marks)
# 5.1 Feature Scaling and Preprocessing pipeline

final_numeric = [c for c in X_train_out.select_dtypes(include=np.number).co
final_categorical = [c for c in X_train_out.columns if c not in final_numer

print('Final numeric features:', final_numeric)
print('Final categorical features:', final_categorical)

numeric_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())
])

# Define a named function for type casting
def cast_to_str(x):
    return x.astype(str)

categorical_transformer = Pipeline(steps=[
    ('type_caster', FunctionTransformer(func=cast_to_str)), # Use the named
    ('imputer', SimpleImputer(strategy='constant', fill_value='missing')),
    ('onehot', OneHotEncoder(handle_unknown='ignore'))
])

preprocessor = ColumnTransformer(transformers=[
    ('num', numeric_transformer, final_numeric),
    ('cat', categorical_transformer, final_categorical)
])
```

```
Final numeric features: ['subtotal', 'num_distinct_items', 'max_item_price',
Final categorical features: ['market_id', 'store_primary_category', 'order_p
```

```
# 5.2 Build LR and perform RFE
lr_pipeline = Pipeline(steps=[('preproc', preprocessor), ('lr', LinearRegre

# Fit baseline
lr_pipeline.fit(X_train_out, y_train)
print('Trained baseline Linear Regression.')

# Baseline performance
y_pred_baseline = lr_pipeline.predict(X_test_out)
print('Baseline performance:')
print('MAE:', mean_absolute_error(y_test, y_pred_baseline))
print('RMSE:', mean_squared_error(y_test, y_pred_baseline))
print('R2:', r2_score(y_test, y_pred_baseline))

# Prepare transformed data for RFE
X_train_transformed = preprocessor.fit_transform(X_train_out)
X_test_transformed = preprocessor.transform(X_test_out)

# feature names
num_names = final_numeric
ohe = preprocessor.named_transformers_['cat'].named_steps['onehot']
cat_names = list(ohe.get_feature_names_out(final_categorical)) if hasattr(o
feature_names = list(num_names) + cat_names
print('Total preprocessed features:', len(feature_names))

# RFE loop
estimator = LinearRegression()
from sklearn.feature_selection import RFE
max_k = min(15, X_train_transformed.shape[1])
results = []
for k in range(5, max_k+1):
    selector = RFE(estimator, n_features_to_select=k, step=1)
    selector = selector.fit(X_train_transformed, y_train)
    sel_idx = selector.get_support(indices=True)
    X_sel = X_train_transformed[:, sel_idx]
    scores = cross_val_score(estimator, X_sel, y_train, cv=5, scoring='neg_
    mean_rmse = -np.mean(scores)
    results.append({'k':k, 'rmse':mean_rmse, 'selected_idx': sel_idx})
    print(f'k={k}, CV RMSE={mean_rmse:.2f} mins')

best = min(results, key=lambda x: x['rmse'])
print('\nBest k:', best['k'], 'RMSE:', best['rmse'])
selected_idx = best['selected_idx']
selected_features = [feature_names[i] for i in selected_idx]
print('\nSelected features:', selected_features)
```

```
Trained baseline Linear Regression.
Baseline performance:
MAE: 2.0779108974683296
RMSE: 8.8637982035397
R2: 0.8987310265930643
Total preprocessed features: 91
k=5, CV RMSE=9.32 mins
```